ID S4

The CAN-BUS and transmission of sensor signals in a CAN network

Prior knowledge

- Temperature sensor with 1-wire interface (ID S1)
- Differential measurements with a two-channel oscilloscope
- CAN bus
- Physical Layer: CANHigh, CANLow, Data Rate
- CAN-Telegram
- Arbitration

Material

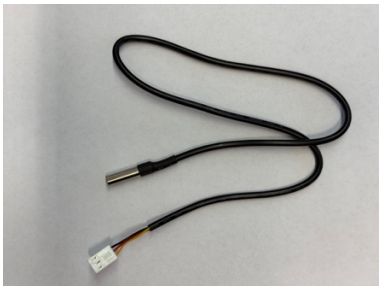
- Material and program code from experiment ID S1 (Arduino, 1-wire temperature sensor)
- USB-CAN Schnittstelle TRU-Components USB-CAN (https://asset.conrad.com/media10/add/160267/c1/-/en/002368701DS00/datenblatt-2368701-tru-components-tc-9474804-can-umsetzer-usb-can-bus-sub-d9-nicht-galvanisch-getrennt-5-vdc-1-st.pdf)
- Male/Male Kabel

Duration approx. 4h

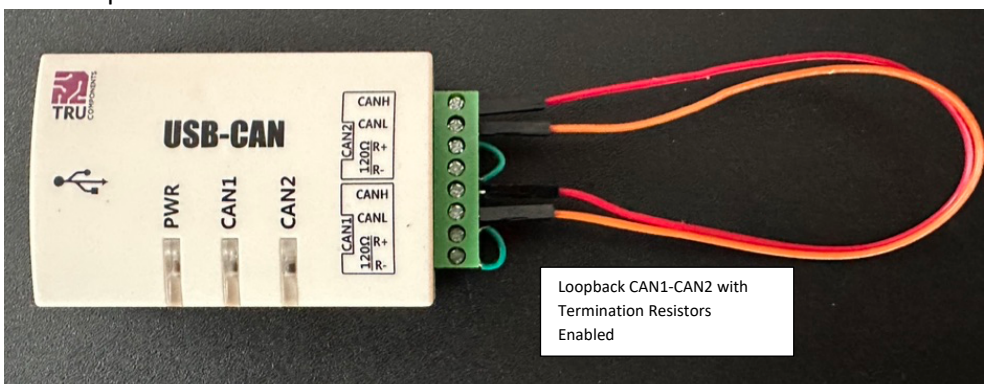
Structure

Hardware:

- 1-wire temperature sensor



- TRU-components USB-CAN



Software:

USB Drivers:

- Windows: Install and connect USB drivers via ZADIG (https://zadig.akeo.ie); connect USB-CAN and map to libusb32; Driver is installed automatically
- OSX: Install HomebrewCommand line:
  - /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
  - echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
  - brew install libusb

For emergencies on Windows:

- https://asset.conrad.com/media10/add/160267/c1/-/gl/002368701DL00/download-2368701-tru-components-tc-9474804-can-umsetzer-usb-can-bus-sub-d9-nicht-galvanisch-getrennt-5-vdc-1-st.zip

Python:

- Anaconda/Spyder with the library "catalystii" (command line: "pip install canalystii")
- Prepared programs from the Teams site (Python)
- Your programs from ID S1 (HCSR04)

Experimental Part 1 – CAN Interface Setup and Test

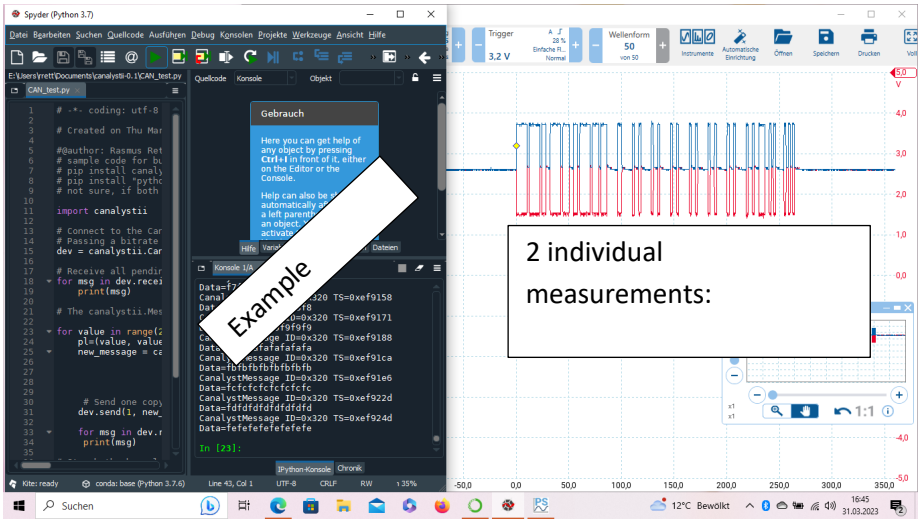
- Install the USB-CAN interface according to the instructions above
- Connect the two CAN interfaces (CANH-CANH and CANL-CANL) and activate the terminators by wire jumpers between R+ and R- for both interfaces
- Download the program "CAN_test.py" and analyze the function. Customize the program as needed. Briefly describe the function. Record the data flow schematically. Change the parameters and payload in the transfer (bitrate; can_id, remote, extended, data_len, data); document your investigations systematically and completely

Deliverables:

- Task in your words
- Photo of the experimental setup
- Flowchart for the program used
- Examination results with the set parameters (see above)

Part 2 – CAN Physical Layer (Loopback CAN1-CAN2)

- Use two channels of the oscilloscope to perform a differential measurement of the voltage between CANH and CANL (with matched probes!)
- Run the program "CAN_test.py" and follow the process on the oscilloscope
- Attach a screendump of a complete CAN telegram to your protocol and label the individual areas of the telegram
- Change the parameter "bitrate" from 500000 to 250000 and make a comparison measurement with the oscilloscope. Document your results with screen dumps and describe the differences.



Deliverables:

- Task in your words
- Photo of the experimental setup
- Screendump of a CAN telegram and several telegrams in one measurement
- Screendump with the name of the individual fields and their content (annotation)
- Screendumps for different settings of the CAN interface

- Examination results with the set parameters (see above)

Test part 3 – Setup of sensor technology and transfer to the CAN bus (loopback CAN1-CAN2)

- Setup the 1-wire temperature sensor according to lab ID-S1. Make sure, the sensor is working properly.
- Modify the python program so that sensor values are first recorded and then transferred to the CAN bus (CAN1). To do this, first create your CAN telegram(s) with the assignment of the bits in the payload. You may need to distribute your payload across multiple telegrams.
- Check whether the sensor data corresponds to the data that is read back on the CAN bus (CAN2).
- Document your tests as examples.
- Record the data flow diagram. How do you bring the data to the CAN bus, how do you read it back

Deliverables:

- Task in your words
- Block diagram of the structure and flow chart of the investigation
- Design & Code: How do you code the measurement result onto the CAN telegram(s)?
- Screendump PC from a measurement, the payload sent and the payload read back. Do both agree?

Experiment Part 4 – Automatic Error Checking (Loopback CAN1-CAN2)

- Automate the process from task part 3 so that deviations are automatically reported (shown on the screen).
- Run the test with at least 100 messages. Are messages lost? Do you observe transmission errors?
- Attach the documented source code to your log.

Deliverables:

- Task in your words
- Block diagram of the structure and flow chart of the investigation
- Design & Code: How do you check the CAN interface automatically?
- Examination result: What error rate were you able to determine? Please find a good graphical representation to show your measurements and detect errors.

Experiment part 5 – Setup of a large CAN network (CAN1 on the common bus, please deactivate all terminators BEFORE)

- Coordinate in the lab: What bitrate do you want to use? Who occupies which identifiers? Document the vote in a spreadsheet.
- Using your python code, describe how the sensor data is encoded in your CAN messages.
- Record the entire CAN network with all participants.
- Transfer your data to the bus and check that at least one group arrives correctly. Perform the test in both directions. Document the results.

Deliverables:

- Task in your words
- Block diagram of the structure and flow chart of the investigation
- Your annotated program code
- Proof of transmission: Screendump PC of a measurement, the sent payload and the read payload. Do both agree?